

ProjetoEDA_F2

Generated by Doxygen 1.13.2

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 Ant Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Field Documentation	5
3.1.2.1 freqAntena	5
3.1.2.2 id	6
3.1.2.3 listaAresta	6
3.1.2.4 proxAntena	6
3.1.2.5 x	6
3.1.2.6 y	6
3.2 Ars Struct Reference	6
3.2.1 Detailed Description	7
3.2.2 Field Documentation	7
3.2.2.1 destinoAntena	7
3.2.2.2 origemAntena	7
3.2.2.3 proximaAresta	7
3.2.2.4 xNef	7
3.2.2.5 yNef	8
3.3 Grafo Struct Reference	8
3.3.1 Detailed Description	8
3.3.2 Field Documentation	8
3.3.2.1 Antena	8
4 File Documentation	9
4.1 projetoEDA_F2/funcoes.c File Reference	9
4.1.1 Function Documentation	10
4.1.1.1 BFS()	10
4.1.1.2 CriarListaArestas()	11
4.1.1.3 DFS()	11
4.1.1.4 EncontrarCaminhos()	12
4.1.1.5 FreeListaAntenas()	12
4.1.1.6 IniciarDFS()	13
4.1.1.7 InserirAntena()	13
4.1.1.8 LerLista()	14
4.1.1.9 TodosCaminhos()	14
4.2 funcoes.c	15
4.3 projetoEDA_F2/funcoes.h File Reference	19

4.3.1 Macro Definition Documentation	20
4.3.1.1 MAX_COLUNAS	20
4.3.1.2 MAX_LINHAS	21
4.3.1.3 MAX_VERTICES	21
4.3.2 Typedef Documentation	21
4.3.2.1 Ant	21
4.3.2.2 Ars	21
4.3.2.3 Grafo	21
4.3.3 Function Documentation	21
4.3.3.1 BFS()	21
4.3.3.2 CriarListaArestas()	22
4.3.3.3 DFS()	22
4.3.3.4 EncontrarCaminhos()	23
4.3.3.5 FreeListaAntenas()	24
4.3.3.6 ImprimirMatriz()	24
4.3.3.7 IniciarDFS()	25
4.3.3.8 InserirAntena()	26
4.3.3.9 LerLista()	27
4.3.3.10 ListarArestasENefastos()	27
4.3.3.11 TodosCaminhos()	28
4.4 funcoes.h	29
4.5 projetoEDA_F2_Exe/funcoes.h File Reference	30
4.5.1 Macro Definition Documentation	31
4.5.1.1 MAX_COLUNAS	31
4.5.1.2 MAX_LINHAS	31
4.5.1.3 MAX_VERTICES	31
4.5.2 Typedef Documentation	32
4.5.2.1 Ant	32
4.5.2.2 Ars	32
4.5.2.3 Grafo	32
4.5.3 Function Documentation	32
4.5.3.1 BFS()	32
4.5.3.2 CriarListaArestas()	32
4.5.3.3 DFS()	33
4.5.3.4 EncontrarCaminhos()	34
4.5.3.5 FreeListaAntenas()	35
4.5.3.6 ImprimirMatriz()	35
4.5.3.7 IniciarDFS()	36
4.5.3.8 InserirAntena()	36
4.5.3.9 LerLista()	37
4.5.3.10 ListarArestasENefastos()	38
4.5.3.11 TodosCaminhos()	39

4.6 funcoes.h	39
4.7 projetoEDA_F2_Exe/apresenta.c File Reference	40
4.7.1 Detailed Description	41
4.7.2 Function Documentation	41
4.7.2.1 ImprimirMatriz()	41
4.7.2.2 ListarArestasENefastos()	42
4.8 apresenta.c	42
4.9 projetoEDA_F2_Exe/main.c File Reference	43
4.9.1 Detailed Description	44
4.9.2 Function Documentation	44
4.9.2.1 main()	44
4.10 main.c	45
Index	47

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

Ant	Representa uma antena (ou vértice) do grafo	5
Ars	Representa uma aresta entre duas antenas, podendo conter a zona nefasta	6
Grafo	Representa o grafo principal através de um array de ponteiros para antenas	8

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

projetoEDA_F2/ funcoes.c	9
projetoEDA_F2/ funcoes.h	19
projetoEDA_F2_Exe/ apresenta.c	
Implementação de funções para apresentação de dados do grafo (matriz e lista de arestas) . .	40
projetoEDA_F2_Exe/ funcoes.h	30
projetoEDA_F2_Exe/ main.c	
Ponto de entrada principal do programa que manipula grafos de antenas	43

Chapter 3

Data Structure Documentation

3.1 Ant Struct Reference

Representa uma antena (ou vértice) do grafo.

```
#include <funcoes.h>
```

Data Fields

- char [freqAntena](#)
- int [id](#)
- int [x](#)
- int [y](#)
- struct [Ant](#) * [proxAntena](#)
- struct [Ars](#) * [listaAresta](#)

3.1.1 Detailed Description

Representa uma antena (ou vértice) do grafo.

Definition at line [29](#) of file [funcoes.h](#).

3.1.2 Field Documentation

3.1.2.1 freqAntena

```
char freqAntena
```

Frequência da antena (ex: '0', 'A', etc).

Definition at line [30](#) of file [funcoes.h](#).

3.1.2.2 id

```
int id
```

Identificador único da antena.

Definition at line 31 of file [funcoes.h](#).

3.1.2.3 listaAresta

```
struct Ars * listaAresta
```

Ponteiro para a lista de arestas associadas a esta antena.

Definition at line 35 of file [funcoes.h](#).

3.1.2.4 proxAntena

```
struct Ant * proxAntena
```

Ponteiro para a próxima antena na lista ligada.

Definition at line 34 of file [funcoes.h](#).

3.1.2.5 x

```
int x
```

Coordenada horizontal (coluna) da antena.

Definition at line 32 of file [funcoes.h](#).

3.1.2.6 y

```
int y
```

Coordenada vertical (linha) da antena.

Definition at line 33 of file [funcoes.h](#).

The documentation for this struct was generated from the following files:

- projetoEDA_F2/[funcoes.h](#)
- projetoEDA_F2_Exe/[funcoes.h](#)

3.2 Ars Struct Reference

Representa uma aresta entre duas antenas, podendo conter a zona nefasta.

```
#include <funcoes.h>
```

Data Fields

- struct [Ant](#) * [origemAntena](#)
- struct [Ant](#) * [destinoAntena](#)
- struct [Ars](#) * [proximaAresta](#)
- int [xNef](#)
- int [yNef](#)

3.2.1 Detailed Description

Representa uma aresta entre duas antenas, podendo conter a zona nefasta.

Definition at line [42](#) of file [funcoes.h](#).

3.2.2 Field Documentation

3.2.2.1 destinoAntena

```
struct Ant * destinoAntena
```

Ponteiro para a antena de destino da ligação.

Definition at line [44](#) of file [funcoes.h](#).

3.2.2.2 origemAntena

```
struct Ant * origemAntena
```

Ponteiro para a antena de origem da ligação.

Definition at line [43](#) of file [funcoes.h](#).

3.2.2.3 proximaAresta

```
struct Ars * proximaAresta
```

Ponteiro para a próxima aresta na lista ligada.

Definition at line [45](#) of file [funcoes.h](#).

3.2.2.4 xNef

```
int xNef
```

Coordenada X da zona nefasta (se existir).

Definition at line [46](#) of file [funcoes.h](#).

3.2.2.5 yNef

```
int yNef
```

Coordenada Y da zona nefasta (se existir).

Definition at line 47 of file [funcoes.h](#).

The documentation for this struct was generated from the following files:

- projetoEDA_F2/[funcoes.h](#)
- projetoEDA_F2_Exe/[funcoes.h](#)

3.3 Grafo Struct Reference

Representa o grafo principal através de um array de ponteiros para antenas.

```
#include <funcoes.h>
```

Data Fields

- struct [Ant](#) * [Antena](#) [[MAX_VERTICES](#)]

3.3.1 Detailed Description

Representa o grafo principal através de um array de ponteiros para antenas.

Definition at line 54 of file [funcoes.h](#).

3.3.2 Field Documentation

3.3.2.1 Antena

```
struct Ant * Antena
```

Array que armazena os ponteiros para antenas indexadas pelo seu ID.

Definition at line 55 of file [funcoes.h](#).

The documentation for this struct was generated from the following files:

- projetoEDA_F2/[funcoes.h](#)
- projetoEDA_F2_Exe/[funcoes.h](#)

Chapter 4

File Documentation

4.1 projetoEDA_F2/funcoes.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "funcoes.h"
```

Functions

- [Ant * LerLista](#) (const char *nomeFicheiro, const char *tipoFicheiro, int *linhas, int *colunas, [Grafo *grafo](#))
Lê a matriz de antenas a partir de um ficheiro (.txt ou .bin) e constrói a lista ligada.
- [Ant * InserirAntena](#) ([Ant *lista](#), [Grafo *grafo](#), char freq, int x, int y, int id)
Inserir uma nova antena na lista ligada e atualiza o grafo com o seu endereço.
- int [CriarListaArestas](#) ([Ant *lista](#), int linhas, int colunas)
Cria as arestas entre antenas com a mesma frequência e calcula zonas nefastas.
- int [DFS](#) ([Ant *atual](#), int visitado[], int idOrigem)
Realiza uma travessia em profundidade (DFS) recursiva a partir de uma antena.
- int [IniciarDFS](#) ([Grafo *grafo](#), int idOrigem)
Função de inicialização para DFS.
- int [BFS](#) ([Grafo *grafo](#), int idOrigem)
Realiza uma travessia em largura (BFS) a partir de uma antena de origem.
- int [EncontrarCaminhos](#) ([Ant *atual](#), int idDestino, int visitado[], int caminho[], int posicao)
Procura todos os caminhos possíveis entre a antena atual e o destino.
- int [TodosCaminhos](#) ([Grafo *grafo](#), int idOrigem, int idDestino)
Função principal para listar todos os caminhos entre duas antenas.
- int [FreeListaAntenas](#) ([Ant *lista](#), [Grafo *grafo](#))
Liberta a memória alocada para a lista de antenas e o grafo.

4.1.1 Function Documentation

4.1.1.1 BFS()

```
int BFS (  
    Grafo * grafo,  
    int idOrigem)
```

Realiza uma travessia em largura (BFS) a partir de uma antena de origem.

Executa a travessia em largura (BFS) a partir de uma antena de origem.

Utiliza uma fila para visitar os vértices por camadas e imprime o percurso.

Parameters

<i>grafo</i>	Grafo que contém todas as antenas
<i>idOrigem</i>	ID da antena de origem

Returns

200 em caso de sucesso, ou se a antena for inválida

Definition at line 302 of file [funcoes.c](#).

4.1.1.2 CriarListaArestas()

```
int CriarListaArestas (  
    Ant * lista,  
    int linhas,  
    int colunas)
```

Cria as arestas entre antenas com a mesma frequência e calcula zonas nefastas.

Cria as arestas entre antenas com base na frequência e calcula zonas nefastas.

Compara todas as antenas entre si e liga aquelas com frequência igual. Cada ligação pode armazenar uma zona nefasta resultante da interferência.

Parameters

<i>lista</i>	Lista ligada com todas as antenas
<i>linhas</i>	Número total de linhas da matriz
<i>colunas</i>	Número total de colunas da matriz

Returns

200 se bem-sucedido, ou 500 em caso de erro de alocação

Definition at line 136 of file [funcoes.c](#).

4.1.1.3 DFS()

```
int DFS (  
    Ant * atual,  
    int visitado[],  
    int idOrigem)
```

Realiza uma travessia em profundidade (DFS) recursiva a partir de uma antena.

Função recursiva que realiza a travessia em profundidade (DFS).

Atravessa todas as conexões do grafo, listando as antenas alcançadas.

Parameters

<i>atual</i>	Ponteiro para a antena atual
<i>visitado</i>	Vetor de controlo de antenas já visitadas
<i>idOrigem</i>	ID da antena anterior (usado para rastrear o percurso)

Returns

Código de estado (200)

Definition at line 244 of file [funcoes.c](#).

4.1.1.4 EncontrarCaminhos()

```
int EncontrarCaminhos (  
    Ant * atual,  
    int idDestino,  
    int visitado[],  
    int caminho[],  
    int posicao)
```

Procura todos os caminhos possíveis entre a antena atual e o destino.

Função recursiva para descobrir todos os caminhos entre duas antenas.

Utiliza backtracking para encontrar todos os caminhos válidos.

Parameters

<i>atual</i>	Ponteiro para a antena atual
<i>idDestino</i>	ID da antena destino
<i>visitado</i>	Vetor de controlo de antenas visitadas
<i>caminho</i>	Vetor para armazenar o caminho atual
<i>posicao</i>	Posição atual no vetor caminho

Returns

200 em caso de sucesso

Definition at line 366 of file [funcoes.c](#).

4.1.1.5 FreeListaAntenas()

```
int FreeListaAntenas (  
    Ant * lista,  
    Grafo * grafo)
```

Liberta a memória alocada para a lista de antenas e o grafo.

Liberta a memória alocada para a lista de antenas e suas arestas.

Percorre a lista de antenas, libertando cada aresta e cada antena. Também limpa o grafo, definindo todos os ponteiros de antena como NULL.

Parameters

<i>lista</i>	Lista ligada de antenas
<i>grafo</i>	Grafo onde as antenas estão registradas

Returns

200 em caso de sucesso

Definition at line [450](#) of file [funcoes.c](#).

4.1.1.6 IniciarDFS()

```
int IniciarDFS (  
    Grafo * grafo,  
    int idOrigem)
```

Função de inicialização para DFS.

Inicia a travessia DFS a partir de uma antena de origem.

Verifica a validade da antena de origem e inicia a travessia.

Parameters

<i>grafo</i>	Grafo onde se realiza a travessia
<i>idOrigem</i>	ID da antena de origem

Returns

200 em caso de sucesso, 500 se o ID for inválido

Definition at line [277](#) of file [funcoes.c](#).

4.1.1.7 InserirAntena()

```
Ant * InserirAntena (  
    Ant * lista,  
    Grafo * grafo,  
    char freq,  
    int x,  
    int y,  
    int id)
```

Insere uma nova antena na lista ligada e atualiza o grafo com o seu endereço.

Insere uma nova antena na lista e no grafo.

Parameters

<i>lista</i>	Lista ligada atual de antenas
<i>grafo</i>	Grafo onde a antena será registrada pelo seu ID
<i>freq</i>	Carácter representativo da frequência da antena
<i>x</i>	Coordenada X (coluna) da antena
<i>y</i>	Coordenada Y (linha) da antena
<i>id</i>	Identificador único da antena

Returns

Ponteiro para a nova cabeça da lista ligada

Definition at line 104 of file [funcoes.c](#).

4.1.1.8 LerLista()

```
Ant * LerLista (
    const char * nomeFicheiro,
    const char * tipoFicheiro,
    int * linhas,
    int * colunas,
    Grafo * grafo)
```

Lê a matriz de antenas a partir de um ficheiro (.txt ou .bin) e constrói a lista ligada.

Lê um ficheiro com a matriz de antenas e cria a lista ligada e o grafo correspondente.

Esta função percorre o ficheiro linha a linha e cria antenas para todas as células não vazias. Após a leitura, chama a função [CriarListaArestas\(\)](#) para estabelecer as ligações entre antenas.

Parameters

<i>nomeFicheiro</i>	Nome base do ficheiro (sem extensão)
<i>tipoFicheiro</i>	Extensão do ficheiro (".txt" ou ".bin")
<i>linhas</i>	Ponteiro para armazenar o número de linhas da matriz
<i>colunas</i>	Ponteiro para armazenar o número de colunas da matriz
<i>grafo</i>	Ponteiro para o grafo onde as antenas serão registadas

Returns

Ponteiro para o início da lista ligada de antenas

Definition at line 33 of file [funcoes.c](#).

4.1.1.9 TodosCaminhos()

```
int TodosCaminhos (
    Grafo * grafo,
    int idOrigem,
    int idDestino)
```

Função principal para listar todos os caminhos entre duas antenas.

Inicia a busca por todos os caminhos possíveis entre duas antenas.

Verifica se as antenas são válidas e têm a mesma frequência antes de iniciar a busca.

Parameters

<i>grafo</i>	Grafo com as antenas
<i>idOrigem</i>	ID da antena de origem
<i>idDestino</i>	ID da antena de destino

Returns

200 em caso de sucesso, ou se os IDs forem inválidos

Definition at line 403 of file [funcoes.c](#).

4.2 funcoes.c

[Go to the documentation of this file.](#)

```

00001
00012
00013
00014 #include <stdio.h>
00015 #include <stdlib.h>
00016 #include <string.h>
00017 #include "funcoes.h"
00018
00019 #pragma region Leitura_Armazenamento
00033 Ant* LerLista(const char* nomeFicheiro, const char* tipoFicheiro, int* linhas, int* colunas, Grafo*
    grafo) {
00034     FILE* ficheiro;
00035     char nomeCompleto[256]; // Buffer para o nome do arquivo
00036     snprintf(nomeCompleto, sizeof(nomeCompleto), "%s%s", nomeFicheiro, tipoFicheiro);
00037     if (strcmp(tipoFicheiro, ".txt") == 0) {
00038         errno_t err = fopen_s(&ficheiro, nomeCompleto, "r");
00039         if (err != 0) {
00040             perror("Erro ao abrir o ficheiro");
00041             return NULL;
00042         }
00043     }
00044     else {
00045         errno_t err = fopen_s(&ficheiro, nomeCompleto, "rb");
00046         if (err != 0) {
00047             perror("Erro ao abrir o ficheiro");
00048             return NULL;
00049         }
00050     }
00051
00052     // Já recebido como argumento
00053     memset(grafo->Antena, 0, sizeof(grafo->Antena)); // Limpa ponteiros
00054     //grafo->Antena[0] = NULL; // Inicializa o primeiro elemento como NULL caso id comece a 1, o [0]
    deve ser null
00055
00056     Ant* lista = NULL;
00057     char linha[MAX_COLUNAS]; //100
00058     int y = 0;
00059     int maxColunas = 0;
00060     int id = 0; // Inicializa o ID da antena
00061
00062     while (fgets(linha, sizeof(linha), ficheiro)) {
00063         int tamLinha = strlen(linha);
00064         if (linha[tamLinha - 1] == '\n') {
00065             linha[tamLinha - 1] = '\0'; // Remover quebra de linha
00066             tamLinha--;
00067         }
00068         if (tamLinha > maxColunas) {
00069             maxColunas = tamLinha;
00070         }
00071         for (int x = 0; x < tamLinha; x++) {
00072             if (linha[x] != '.') {
00073                 lista = InserirAntena(lista, grafo, linha[x], x, y, id);
00074                 id++; // Incrementa o ID da antena
00075             }
00076         }
00077         y++;
00078     }

```

```

00079
00080     *linhas = y;
00081     *colunas = maxColunas;
00082
00083     int verificaArestas = CriarListaArestas(lista, y, maxColunas);
00084     if (verificaArestas == 0) {
00085         perror("Erro ao criar lista de arestas");
00086         fclose(ficheiro);
00087         return NULL;
00088     }
00089     fclose(ficheiro);
00090     return lista;
00091 }
00092
00104 Ant* InserirAntena(Ant* lista, Grafo* grafo, char freq, int x, int y, int id) {
00105     Ant* novaAntena = (Ant*)malloc(sizeof(Ant));
00106     if (novaAntena == NULL) {
00107         perror("Erro ao alocar memória para antena");
00108         return lista;
00109     }
00110
00111     novaAntena->freqAntena = freq;
00112     novaAntena->x = x;
00113     novaAntena->y = y;
00114     novaAntena->id = id;
00115     novaAntena->proxAntena = lista;
00116     novaAntena->listaAresta = NULL;
00117
00118     if (grafo != NULL && id < MAX_VERTICES) {
00119         grafo->Antena[id] = novaAntena;
00120     }
00121
00122     return novaAntena;
00123 }
00124
00136 int CriarListaArestas(Ant* lista, int linhas, int colunas) {
00137     int menorx, memory, maiorx, maiory, difx, dify, idant1, idant2;
00138     Ant* listaAnt1 = lista;
00139     while (listaAnt1 != NULL) {
00140         Ant* listaAnt2 = lista;
00141
00142         while (listaAnt2 != NULL) {
00143
00144             if (listaAnt1->freqAntena == listaAnt2->freqAntena && (listaAnt1->y != listaAnt2->y ||
listaAnt1->x != listaAnt2->x)) {
00145                 Ars* novaAresta = (Ars*)malloc(sizeof(Ars));
00146                 if (novaAresta == NULL) {
00147                     perror("Erro ao alocar memoria para a aresta");
00148                     return 500;
00149                 }
00150                 novaAresta->origemAntena = listaAnt1;
00151                 novaAresta->destinoAntena = listaAnt2;
00152                 novaAresta->proximaAresta = listaAnt1->listaAresta;
00153                 listaAnt1->listaAresta = novaAresta;
00154
00155                 if (listaAnt1->y != listaAnt2->y || listaAnt1->x != listaAnt2->x){
00156                     if (listaAnt1->x > listaAnt2->x)
00157                     {
00158                         difx = listaAnt1->x - listaAnt2->x;
00159                         menorx = listaAnt2->x - difx;
00160                         maiorx = listaAnt1->x + difx;
00161                     }
00162                     else
00163                     {
00164                         difx = listaAnt2->x - listaAnt1->x;
00165                         menorx = listaAnt1->x - difx;
00166                         maiorx = listaAnt2->x + difx;
00167                     }
00168                     if (listaAnt1->y > listaAnt2->y)
00169                     {
00170                         dify = listaAnt1->y - listaAnt2->y;
00171                         memory = listaAnt2->y - dify;
00172                         maiory = listaAnt1->y + dify;
00173                     }
00174                     else
00175                     {
00176                         dify = listaAnt2->y - listaAnt1->y;
00177                         memory = listaAnt1->y - dify;
00178                         maiory = listaAnt2->y + dify;
00179                     }
00180
00181                     idant1 = listaAnt1->id;
00182                     idant2 = listaAnt2->id;
00183
00184                     if (listaAnt1->x > listaAnt2->x && listaAnt1->y > listaAnt2->y || listaAnt1->x <
listaAnt2->x && listaAnt1->y < listaAnt2->y)
00185                 {

```

```

00186         if (listaAnt1->y < listaAnt2->y) {
00187             if (menorx >= 0 && memory >= 0 && menorx <= linhas && memory <= colunas)
00188             {
00189                 novaAresta->xNef = menorx;
00190                 novaAresta->yNef = memory;
00191             }
00192         }
00193         else {
00194             if (maiorx <= colunas && maiory <= linhas && maiorx <= linhas && maiory <=
colunas)
00195             {
00196                 novaAresta->xNef = maiorx;
00197                 novaAresta->yNef = maiory;
00198             }
00199         }
00200     }
00201     else if (listaAnt1->x > listaAnt2->x && listaAnt1->y < listaAnt2->y ||
listaAnt1->x < listaAnt2->x && listaAnt1->y > listaAnt2->y)
00202     {
00203         if (listaAnt1->y < listaAnt2->y) {
00204             if (maiorx >= 0 && memory >= 0 && maiorx <= linhas && memory <= colunas)
00205             {
00206                 novaAresta->xNef = maiorx;
00207                 novaAresta->yNef = memory;
00208             }
00209         }
00210         else {
00211             if (menorx <= colunas && maiory <= linhas && menorx <= linhas && maiory <=
colunas)
00212             {
00213                 novaAresta->xNef = menorx;
00214                 novaAresta->yNef = maiory;
00215             }
00216         }
00217     }
00218 }
00219 }
00220 }
00221 }
00222     listaAnt2 = listaAnt2->proxAntena;
00223 }
00224     listaAnt1 = listaAnt1->proxAntena;
00225 }
00226     return 200;
00227 }
00228 }
00229 }
00230 #pragma endregion
00231 }
00232 }
00233 #pragma region DFS
00244 int DFS(Ant* atual, int visitado[], int idOrigem) {
00245     if (atual == NULL || visitado[atual->id]) {
00246         return 200;
00247     }
00248     visitado[atual->id] = 1;
00249     if (idOrigem == -1) {
00250         printf("Antena %d (%c) em [%d, %d] (Origem)\n",
atual->id, atual->freqAntena, atual->x, atual->y);
00251     }
00252     else {
00253         printf("Antena %d (%c) em [%d, %d] (Aresta: origem %d -> destino %d)\n",
atual->id, atual->freqAntena, atual->x, atual->y, idOrigem, atual->id);
00254     }
00255     Ars* aresta = atual->listaAresta;
00256     while (aresta != NULL) {
00257         DFS(aresta->destinoAntena, visitado, atual->id);
00258         aresta = aresta->proximaAresta;
00259     }
00260     return 200;
00261 }
00262 }
00263 }
00264 }
00265 }
00266 }
00267 }
00277 int IniciarDFS(Grafo* grafo, int idOrigem) {
00278     if (idOrigem < 0 || idOrigem >= MAX_VERTICES || grafo->Antena[idOrigem] == NULL) {
00279         fprintf(stderr, "Antena com ID %d não existe.\n", idOrigem);
00280         return 500;
00281     }
00282     int visitado[MAX_VERTICES] = { 0 };
00283     printf("DFS a partir da antena %d:\n", idOrigem);
00284     DFS(grafo->Antena[idOrigem], visitado, -1); // -1 = raiz
00285     printf("\n\n");
00286     return 200;
00287 }
00288 }

```

```

00289 #pragma endregion
00290
00291
00292 #pragma region BFS
00302 int BFS(Grafo* grafo, int idOrigem) {
00303     if (grafo->Antena[idOrigem] == NULL) {
00304         printf("Antena com ID %d não existe.\n", idOrigem);
00305         return 200;
00306     }
00307
00308     int visitado[MAX_VERTICES] = { 0 };
00309     int fila[MAX_VERTICES];
00310     int origem[MAX_VERTICES]; // Guarda de onde veio cada antena
00311
00312     int frente = 0, tras = 0;
00313     fila[tras] = idOrigem;
00314     origem[tras] = -1;
00315     visitado[idOrigem] = 1;
00316
00317     printf("BFS a partir da antena %d:\n", idOrigem);
00318
00319     while (frente <= tras) {
00320         int atualID = fila[frente];
00321         int origemID = origem[frente];
00322         frente++;
00323
00324         Ant* atual = grafo->Antena[atualID];
00325
00326         if (origemID == -1) {
00327             printf("Antena %d (%c) em [%d, %d] (Origem)\n",
00328                 atual->id, atual->freqAntena, atual->x, atual->y);
00329         }
00330         else {
00331             printf("Antena %d (%c) em [%d, %d] (Aresta: origem %d -> destino %d)\n",
00332                 atual->id, atual->freqAntena, atual->x, atual->y, origemID, atualID);
00333         }
00334
00335         Ars* aresta = atual->listaAresta;
00336         while (aresta != NULL) {
00337             int vizinhoID = aresta->destinoAntena->id;
00338             if (!visitado[vizinhoID]) {
00339                 tras++;
00340                 fila[tras] = vizinhoID;
00341                 origem[tras] = atualID;
00342                 visitado[vizinhoID] = 1;
00343             }
00344             aresta = aresta->proximaAresta;
00345         }
00346     }
00347     printf("\n\n");
00348     return 200;
00349 }
00350 #pragma endregion
00351
00352 #pragma region Todos_Caminhos
00366 int EncontrarCaminhos(Ant* atual, int idDestino, int visitado[], int caminho[], int posicao) {
00367     if (atual == NULL) return 200;
00368
00369     visitado[atual->id] = 1;
00370     caminho[posicao++] = atual->id;
00371
00372     if (atual->id == idDestino) {
00373         for (int i = 0; i < posicao; i++) {
00374             printf("%d", caminho[i]);
00375             if (i < posicao - 1) printf(" -> ");
00376         }
00377         printf("\n");
00378     }
00379     else {
00380         Ars* aresta = atual->listaAresta;
00381         while (aresta != NULL) {
00382             int vizinhoID = aresta->destinoAntena->id;
00383             if (!visitado[vizinhoID]) {
00384                 EncontrarCaminhos(aresta->destinoAntena, idDestino, visitado, caminho, posicao);
00385             }
00386             aresta = aresta->proximaAresta;
00387         }
00388     }
00389
00390     visitado[atual->id] = 0; // backtracking
00391     return 200;
00392 }
00403 int TodosCaminhos(Grafo* grafo, int idOrigem, int idDestino) {
00404     if (grafo->Antena[idOrigem] == NULL || grafo->Antena[idDestino] == NULL) {
00405         printf("Antena(s) inválida(s)\n");
00406         return 200;

```



```

00407     }
00408
00409     int visitado[MAX_VERTICES] = { 0 };
00410     int caminho[MAX_VERTICES];
00411     printf("Todos os caminhos de %d para %d:\n", idOrigem, idDestino);
00412     //compara a frequencia da antena origem e destino
00413     if (grafo->Antena[idOrigem]->freqAntena != grafo->Antena[idDestino]->freqAntena) {
00414         printf("As antenas de origem e destino nao tem a mesma frequencia.\n\n");
00415         return 200;
00416     }
00417     EncontrarCaminhos(grafo->Antena[idOrigem], idDestino, visitado, caminho, 0);
00418     printf("\n\n");
00419     return 200;
00420 }
00421 }
00422 #pragma endregion
00423
00424
00425
00426
00427
00428
00429
00430
00431
00432
00433
00434
00435
00436
00437
00438
00439 #pragma region Libertacao_Memoria
00450 int FreeListaAntenas(Ant* lista, Grafo* grafo) {
00451     Ant* atual = lista;
00452     while (atual != NULL) {
00453         Ars* aresta = atual->listaAresta;
00454         // Liberta todas as arestas desta antena
00455         while (aresta != NULL) {
00456             Ars* tempAresta = aresta;
00457             aresta = aresta->proximaAresta;
00458             free(tempAresta);
00459         }
00460
00461         Ant* tempAntena = atual;
00462         atual = atual->proxAntena;
00463         free(tempAntena);
00464     }
00465     for (int i = 0; i < MAX_VERTICES; i++) {
00466         grafo->Antena[i] = NULL;
00467     }
00468
00469     return 200;
00470 }
00471
00472 #pragma endregion
00473
00474
00475
00476

```

4.3 projetoEDA_F2/funcoes.h File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Data Structures

- struct [Ant](#)
Representa uma antena (ou vértice) do grafo.
- struct [Ars](#)
Representa uma aresta entre duas antenas, podendo conter a zona nefasta.
- struct [Grafo](#)
Representa o grafo principal através de um array de ponteiros para antenas.

Macros

- `#define` [MAX_LINHAS](#) 100
- `#define` [MAX_COLUNAS](#) 100
- `#define` [MAX_VERTICES](#) 200

Typedefs

- `typedef struct` [Ant](#) [Ant](#)
- `typedef struct` [Ars](#) [Ars](#)
- `typedef struct` [Grafo](#) [Grafo](#)

Functions

- [Ant](#) * [LerLista](#) (const char *nomeFicheiro, const char *tipoFicheiro, int *linhas, int *colunas, [Grafo](#) *grafo)
Lê um ficheiro com a matriz de antenas e cria a lista ligada e o grafo correspondente.
- [Ant](#) * [InserirAntena](#) ([Ant](#) *lista, [Grafo](#) *grafo, char freq, int x, int y, int id)
Insere uma nova antena na lista e no grafo.
- int [CriarListaArestas](#) ([Ant](#) *lista, int linhas, int colunas)
Cria as arestas entre antenas com base na frequência e calcula zonas nefastas.
- int [DFS](#) ([Ant](#) *atual, int visitado[], int idOrigem)
Função recursiva que realiza a travessia em profundidade (DFS).
- int [IniciarDFS](#) ([Grafo](#) *grafo, int idOrigem)
Inicia a travessia DFS a partir de uma antena de origem.
- int [BFS](#) ([Grafo](#) *grafo, int idOrigem)
Executa a travessia em largura (BFS) a partir de uma antena de origem.
- int [EncontrarCaminhos](#) ([Ant](#) *atual, int idDestino, int visitado[], int caminho[], int posicao)
Função recursiva para descobrir todos os caminhos entre duas antenas.
- int [TodosCaminhos](#) ([Grafo](#) *grafo, int idOrigem, int idDestino)
Inicia a busca por todos os caminhos possíveis entre duas antenas.
- int [ImprimirMatriz](#) ([Grafo](#) *grafo, int linhas, int colunas)
Imprime a matriz com a posição das antenas.
- int [ListarArestasENefastos](#) ([Grafo](#) *grafo, int linhas, int colunas)
Lista todas as arestas existentes e suas zonas nefastas (se existirem).
- int [FreeListaAntenas](#) ([Ant](#) *lista, [Grafo](#) *grafo)
Liberta a memória alocada para a lista de antenas e suas arestas.

4.3.1 Macro Definition Documentation

4.3.1.1 MAX_COLUNAS

```
#define MAX_COLUNAS 100
```

Número máximo de colunas da matriz.

Definition at line 20 of file [funcoes.h](#).

4.3.1.2 MAX_LINHAS

```
#define MAX_LINHAS 100
```

Número máximo de linhas da matriz.

Definition at line 19 of file [funcoes.h](#).

4.3.1.3 MAX_VERTICES

```
#define MAX_VERTICES 200
```

Número máximo de antenas (vértices) no grafo.

Definition at line 21 of file [funcoes.h](#).

4.3.2 Typedef Documentation

4.3.2.1 Ant

```
typedef struct Ant Ant
```

4.3.2.2 Ars

```
typedef struct Ars Ars
```

4.3.2.3 Grafo

```
typedef struct Grafo Grafo
```

4.3.3 Function Documentation

4.3.3.1 BFS()

```
int BFS (  
    Grafo * grafo,  
    int idOrigem)
```

Executa a travessia em largura (BFS) a partir de uma antena de origem.

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>idOrigem</i>	ID da antena de origem.

Returns

200 em caso de sucesso.

Executa a travessia em largura (BFS) a partir de uma antena de origem.

Utiliza uma fila para visitar os vértices por camadas e imprime o percurso.

Parameters

<i>grafo</i>	Grafo que contém todas as antenas
<i>idOrigem</i>	ID da antena de origem

Returns

200 em caso de sucesso, ou se a antena for inválida

Definition at line 302 of file [funcoes.c](#).

4.3.3.2 CriarListaArestas()

```
int CriarListaArestas (  
    Ant * lista,  
    int linhas,  
    int colunas)
```

Cria as arestas entre antenas com base na frequência e calcula zonas nefastas.

Parameters

<i>lista</i>	Lista de antenas.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso, ou 500 em caso de erro de alocação.

Cria as arestas entre antenas com base na frequência e calcula zonas nefastas.

Compara todas as antenas entre si e liga aquelas com frequência igual. Cada ligação pode armazenar uma zona nefasta resultante da interferência.

Parameters

<i>lista</i>	Lista ligada com todas as antenas
<i>linhas</i>	Número total de linhas da matriz
<i>colunas</i>	Número total de colunas da matriz

Returns

200 se bem-sucedido, ou 500 em caso de erro de alocação

Definition at line 136 of file [funcoes.c](#).

4.3.3.3 DFS()

```
int DFS (  
    Ant * atual,  
    int visitado[],  
    int idOrigem)
```

Função recursiva que realiza a travessia em profundidade (DFS).

Parameters

<i>atual</i>	Ponteiro para a antena atual.
<i>visitado</i>	Vetor de controlo de visitados.
<i>idOrigem</i>	ID da antena anterior (para rastreamento).

Returns

200 em caso de sucesso.

Função recursiva que realiza a travessia em profundidade (DFS).

Atravessa todas as conexões do grafo, listando as antenas alcançadas.

Parameters

<i>atual</i>	Ponteiro para a antena atual
<i>visitado</i>	Vetor de controlo de antenas já visitadas
<i>idOrigem</i>	ID da antena anterior (usado para rastrear o percurso)

Returns

Código de estado (200)

Definition at line 244 of file [funcoes.c](#).

4.3.3.4 EncontrarCaminhos()

```
int EncontrarCaminhos (  
    Ant * atual,  
    int idDestino,  
    int visitado[],  
    int caminho[],  
    int posicao)
```

Função recursiva para descobrir todos os caminhos entre duas antenas.

Parameters

<i>atual</i>	Antena atual no percurso.
<i>idDestino</i>	ID da antena destino.
<i>visitado</i>	Vetor de controlo de visitados.
<i>caminho</i>	Vetor para armazenar o caminho atual.
<i>posicao</i>	Posição atual no vetor de caminho.

Returns

200 em caso de sucesso.

Função recursiva para descobrir todos os caminhos entre duas antenas.

Utiliza backtracking para encontrar todos os caminhos válidos.

Parameters

<i>atual</i>	Ponteiro para a antena atual
<i>idDestino</i>	ID da antena destino
<i>visitado</i>	Vetor de controlo de antenas visitadas
<i>caminho</i>	Vetor para armazenar o caminho atual
<i>posicao</i>	Posição atual no vetor caminho

Returns

200 em caso de sucesso

Definition at line 366 of file [funcoes.c](#).

4.3.3.5 FreeListaAntenas()

```
int FreeListaAntenas (  
    Ant * lista,  
    Grafo * grafo)
```

Liberta a memória alocada para a lista de antenas e suas arestas.

Parameters

<i>lista</i>	Lista de antenas.
<i>grafo</i>	Grafo a ser limpo.

Returns

200 após a libertação completa.

Liberta a memória alocada para a lista de antenas e suas arestas.

Percorre a lista de antenas, libertando cada aresta e cada antena. Também limpa o grafo, definindo todos os ponteiros de antena como NULL.

Parameters

<i>lista</i>	Lista ligada de antenas
<i>grafo</i>	Grafo onde as antenas estão registadas

Returns

200 em caso de sucesso

Definition at line 450 of file [funcoes.c](#).

4.3.3.6 ImprimirMatriz()

```
int ImprimirMatriz (  
    Grafo * grafo,  
    int linhas,  
    int colunas)
```

Imprime a matriz com a posição das antenas.

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso.

Imprime a matriz com a posição das antenas.

A matriz representa:

- Cada antena com a sua frequência (ex: '0', 'A', etc.);
- Cada zona nefasta com o símbolo '#';
- Células vazias com o símbolo '.'.

Parameters

<i>grafo</i>	Ponteiro para o grafo que contém as antenas.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso.

Definition at line 32 of file [apresenta.c](#).

4.3.3.7 IniciarDFS()

```
int IniciarDFS (  
    Grafo * grafo,  
    int idOrigem)
```

Inicia a travessia DFS a partir de uma antena de origem.

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>idOrigem</i>	ID da antena de origem.

Returns

200 se bem-sucedido, 500 se o ID for inválido.

Inicia a travessia DFS a partir de uma antena de origem.

Verifica a validade da antena de origem e inicia a travessia.

Parameters

<i>grafo</i>	Grafo onde se realiza a travessia
<i>idOrigem</i>	ID da antena de origem

Returns

200 em caso de sucesso, 500 se o ID for inválido

Definition at line 277 of file [funcoes.c](#).

4.3.3.8 InserirAntena()

```
Ant * InserirAntena (
    Ant * lista,
    Grafo * grafo,
    char freq,
    int x,
    int y,
    int id)
```

Insere uma nova antena na lista e no grafo.

Parameters

<i>lista</i>	Lista atual de antenas.
<i>grafo</i>	Ponteiro para o grafo.
<i>freq</i>	Carácter da frequência da antena.
<i>x</i>	Coordenada X da antena.
<i>y</i>	Coordenada Y da antena.
<i>id</i>	Identificador único da antena.

Returns

Ponteiro para a nova cabeça da lista ligada.

Insere uma nova antena na lista e no grafo.

Parameters

<i>lista</i>	Lista ligada atual de antenas
<i>grafo</i>	Grafo onde a antena será registada pelo seu ID
<i>freq</i>	Carácter representativo da frequência da antena
<i>x</i>	Coordenada X (coluna) da antena
<i>y</i>	Coordenada Y (linha) da antena
<i>id</i>	Identificador único da antena

Returns

Ponteiro para a nova cabeça da lista ligada

Definition at line 104 of file [funcoes.c](#).

4.3.3.9 LerLista()

```
Ant * LerLista (
    const char * nomeFicheiro,
    const char * tipoFicheiro,
    int * linhas,
    int * colunas,
    Grafo * grafo)
```

Lê um ficheiro com a matriz de antenas e cria a lista ligada e o grafo correspondente.

Parameters

<i>nomeFicheiro</i>	Nome base do ficheiro (sem extensão).
<i>tipoFicheiro</i>	Extensão do ficheiro (".txt" ou ".bin").
<i>linhas</i>	Ponteiro onde será armazenado o número de linhas lidas.
<i>colunas</i>	Ponteiro onde será armazenado o número de colunas lidas.
<i>grafo</i>	Ponteiro para a estrutura do grafo a ser preenchida.

Returns

Ponteiro para o início da lista ligada de antenas.

Lê um ficheiro com a matriz de antenas e cria a lista ligada e o grafo correspondente.

Esta função percorre o ficheiro linha a linha e cria antenas para todas as células não vazias. Após a leitura, chama a função [CriarListaArestas\(\)](#) para estabelecer as ligações entre antenas.

Parameters

<i>nomeFicheiro</i>	Nome base do ficheiro (sem extensão)
<i>tipoFicheiro</i>	Extensão do ficheiro (".txt" ou ".bin")
<i>linhas</i>	Ponteiro para armazenar o número de linhas da matriz
<i>colunas</i>	Ponteiro para armazenar o número de colunas da matriz
<i>grafo</i>	Ponteiro para o grafo onde as antenas serão registadas

Returns

Ponteiro para o início da lista ligada de antenas

Definition at line 33 of file [funcoes.c](#).

4.3.3.10 ListarArestasENefastos()

```
int ListarArestasENefastos (
    Grafo * grafo,
    int linhas,
    int colunas)
```

Lista todas as arestas existentes e suas zonas nefastas (se existirem).

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso.

Lista todas as arestas existentes e suas zonas nefastas (se existirem).

Para cada antena no grafo:

- Mostra a sua frequência e posição;
- Lista todas as arestas para antenas com mesma frequência;
- Indica a coordenada da zona nefasta associada, se válida;

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>linhas</i>	Número total de linhas da matriz (para validar nefastos).
<i>colunas</i>	Número total de colunas da matriz (para validar nefastos).

Returns

200 em caso de sucesso.

Definition at line 85 of file [apresenta.c](#).

4.3.3.11 TodosCaminhos()

```
int TodosCaminhos (  
    Grafo * grafo,  
    int idOrigem,  
    int idDestino)
```

Inicia a busca por todos os caminhos possíveis entre duas antenas.

Parameters

<i>grafo</i>	Grafo com todas as antenas.
<i>idOrigem</i>	ID da antena de origem.
<i>idDestino</i>	ID da antena de destino.

Returns

200 em caso de sucesso.

Inicia a busca por todos os caminhos possíveis entre duas antenas.

Verifica se as antenas são válidas e têm a mesma frequência antes de iniciar a busca.

Parameters

<i>grafo</i>	Grafo com as antenas
<i>idOrigem</i>	ID da antena de origem
<i>idDestino</i>	ID da antena de destino

Returns

200 em caso de sucesso, ou se os IDs forem inválidos

Definition at line 403 of file [funcoes.c](#).

4.4 funcoes.h

[Go to the documentation of this file.](#)

```
00001
00013
00014 #pragma once
00015 #include <stdio.h>
00016 #include <stdlib.h>
00017 #include <string.h>
00018
00019 #define MAX_LINHAS 100
00020 #define MAX_COLUNAS 100
00021 #define MAX_VERTICES 200
00022
00023 #pragma region Estrutura_grafo
00024
00029 typedef struct Ant {
00030     char freqAntena;
00031     int id;
00032     int x;
00033     int y;
00034     struct Ant* proxAntena;
00035     struct Ars* listaAresta;
00036 } Ant;
00037
00042 typedef struct Ars {
00043     struct Ant* origemAntena;
00044     struct Ant* destinoAntena;
00045     struct Ars* proximaAresta;
00046     int xNef;
00047     int yNef;
00048 } Ars;
00049
00054 typedef struct Grafo {
00055     struct Ant* Antena[MAX_VERTICES];
00056 } Grafo;
00057
00058 #pragma endregion
00059
00060 #pragma region Leitura_Armazenamento
00061
00072 Ant* LerLista(const char* nomeFicheiro, const char* tipoFicheiro, int* linhas, int* colunas, Grafo*
    grafo);
00073
00085 Ant* InserirAntena(Ant* lista, Grafo* grafo, char freq, int x, int y, int id);
00086
00095 int CriarListaArestas(Ant* lista, int linhas, int colunas);
00096
00097 #pragma endregion
00098
00099 #pragma region DFS
00100
00109 int DFS(Ant* atual, int visitado[], int idOrigem);
00110
00118 int IniciarDFS(Grafo* grafo, int idOrigem);
00119
00120 #pragma endregion
00121
00122 #pragma region BFS
00123
```

```
00131 int BFS(Grafo* grafo, int idOrigem);
00132
00133 #pragma endregion
00134
00135 #pragma region Todos_Caminhos
00136
00147 int EncontrarCaminhos(Ant* atual, int idDestino, int visitado[], int caminho[], int posicao);
00148
00157 int TodosCaminhos(Grafo* grafo, int idOrigem, int idDestino);
00158
00159 #pragma endregion
00160
00161 #pragma region Impressao
00162
00171 int ImprimirMatriz(Grafo* grafo, int linhas, int colunas);
00172
00181 int ListarArestasENefastos(Grafo* grafo, int linhas, int colunas);
00182
00183 #pragma endregion
00184
00185 #pragma region Libertacao_Memoria
00186
00194 int FreeListaAntenas(Ant* lista, Grafo* grafo);
00195
00196 #pragma endregion
```

4.5 projetoEDA_F2_Exe/funcoes.h File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Data Structures

- struct [Ant](#)
Representa uma antena (ou vértice) do grafo.
- struct [Ars](#)
Representa uma aresta entre duas antenas, podendo conter a zona nefasta.
- struct [Grafo](#)
Representa o grafo principal através de um array de ponteiros para antenas.

Macros

- #define [MAX_LINHAS](#) 100
- #define [MAX_COLUNAS](#) 100
- #define [MAX_VERTICES](#) 200

Typedefs

- typedef struct Ant [Ant](#)
- typedef struct Ars [Ars](#)
- typedef struct Grafo [Grafo](#)

Functions

- [Ant * LerLista](#) (const char *nomeFicheiro, const char *tipoFicheiro, int *linhas, int *colunas, [Grafo *grafo](#))
Lê um ficheiro com a matriz de antenas e cria a lista ligada e o grafo correspondente.
- [Ant * InserirAntena](#) ([Ant *lista](#), [Grafo *grafo](#), char freq, int x, int y, int id)
Inserir uma nova antena na lista e no grafo.
- int [CriarListaArestas](#) ([Ant *lista](#), int linhas, int colunas)
Cria as arestas entre antenas com base na frequência e calcula zonas nefastas.
- int [DFS](#) ([Ant *atual](#), int visitado[], int idOrigem)
Função recursiva que realiza a travessia em profundidade (DFS).
- int [IniciarDFS](#) ([Grafo *grafo](#), int idOrigem)
Inicia a travessia DFS a partir de uma antena de origem.
- int [BFS](#) ([Grafo *grafo](#), int idOrigem)
Executa a travessia em largura (BFS) a partir de uma antena de origem.
- int [EncontrarCaminhos](#) ([Ant *atual](#), int idDestino, int visitado[], int caminho[], int posicao)
Função recursiva para descobrir todos os caminhos entre duas antenas.
- int [TodosCaminhos](#) ([Grafo *grafo](#), int idOrigem, int idDestino)
Inicia a busca por todos os caminhos possíveis entre duas antenas.
- int [ImprimirMatriz](#) ([Grafo *grafo](#), int linhas, int colunas)
Imprime a matriz com a posição das antenas.
- int [ListarArestasENefastos](#) ([Grafo *grafo](#), int linhas, int colunas)
Lista todas as arestas existentes e suas zonas nefastas (se existirem).
- int [FreeListaAntenas](#) ([Ant *lista](#), [Grafo *grafo](#))
Liberta a memória alocada para a lista de antenas e suas arestas.

4.5.1 Macro Definition Documentation

4.5.1.1 MAX_COLUNAS

```
#define MAX_COLUNAS 100
```

Número máximo de colunas da matriz.

Definition at line 20 of file [funcoes.h](#).

4.5.1.2 MAX_LINHAS

```
#define MAX_LINHAS 100
```

Número máximo de linhas da matriz.

Definition at line 19 of file [funcoes.h](#).

4.5.1.3 MAX_VERTICES

```
#define MAX_VERTICES 200
```

Número máximo de antenas (vértices) no grafo.

Definition at line 21 of file [funcoes.h](#).

4.5.2 Typedef Documentation

4.5.2.1 Ant

```
typedef struct Ant Ant
```

4.5.2.2 Ars

```
typedef struct Ars Ars
```

4.5.2.3 Grafo

```
typedef struct Grafo Grafo
```

4.5.3 Function Documentation

4.5.3.1 BFS()

```
int BFS (
    Grafo * grafo,
    int idOrigem)
```

Executa a travessia em largura (BFS) a partir de uma antena de origem.

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>idOrigem</i>	ID da antena de origem.

Returns

200 em caso de sucesso.

Executa a travessia em largura (BFS) a partir de uma antena de origem.

Utiliza uma fila para visitar os vértices por camadas e imprime o percurso.

Parameters

<i>grafo</i>	Grafo que contém todas as antenas
<i>idOrigem</i>	ID da antena de origem

Returns

200 em caso de sucesso, ou se a antena for inválida

Definition at line [302](#) of file [funcoes.c](#).

4.5.3.2 CriarListaArestas()

```
int CriarListaArestas (
    Ant * lista,
    int linhas,
    int colunas)
```

Cria as arestas entre antenas com base na frequência e calcula zonas nefastas.

Parameters

<i>lista</i>	Lista de antenas.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso, ou 500 em caso de erro de alocação.

Cria as arestas entre antenas com base na frequência e calcula zonas nefastas.

Compara todas as antenas entre si e liga aquelas com frequência igual. Cada ligação pode armazenar uma zona nefasta resultante da interferência.

Parameters

<i>lista</i>	Lista ligada com todas as antenas
<i>linhas</i>	Número total de linhas da matriz
<i>colunas</i>	Número total de colunas da matriz

Returns

200 se bem-sucedido, ou 500 em caso de erro de alocação

Definition at line 136 of file [funcoes.c](#).

4.5.3.3 DFS()

```
int DFS (  
    Ant * atual,  
    int visitado[],  
    int idOrigem)
```

Função recursiva que realiza a travessia em profundidade (DFS).

Parameters

<i>atual</i>	Ponteiro para a antena atual.
<i>visitado</i>	Vetor de controlo de visitados.
<i>idOrigem</i>	ID da antena anterior (para rastreamento).

Returns

200 em caso de sucesso.

Função recursiva que realiza a travessia em profundidade (DFS).

Atravessa todas as conexões do grafo, listando as antenas alcançadas.

Parameters

<i>atual</i>	Ponteiro para a antena atual
<i>visitado</i>	Vetor de controlo de antenas já visitadas
<i>idOrigem</i>	ID da antena anterior (usado para rastrear o percurso)

Returns

Código de estado (200)

Definition at line 244 of file [funcoes.c](#).

4.5.3.4 EncontrarCaminhos()

```
int EncontrarCaminhos (  
    Ant * atual,  
    int idDestino,  
    int visitado[],  
    int caminho[],  
    int posicao)
```

Função recursiva para descobrir todos os caminhos entre duas antenas.

Parameters

<i>atual</i>	Antena atual no percurso.
<i>idDestino</i>	ID da antena destino.
<i>visitado</i>	Vetor de controlo de visitados.
<i>caminho</i>	Vetor para armazenar o caminho atual.
<i>posicao</i>	Posição atual no vetor de caminho.

Returns

200 em caso de sucesso.

Função recursiva para descobrir todos os caminhos entre duas antenas.

Utiliza backtracking para encontrar todos os caminhos válidos.

Parameters

<i>atual</i>	Ponteiro para a antena atual
<i>idDestino</i>	ID da antena destino
<i>visitado</i>	Vetor de controlo de antenas visitadas
<i>caminho</i>	Vetor para armazenar o caminho atual
<i>posicao</i>	Posição atual no vetor caminho

Returns

200 em caso de sucesso

Definition at line 366 of file [funcoes.c](#).

4.5.3.5 FreeListaAntenas()

```
int FreeListaAntenas (  
    Ant * lista,  
    Grafo * grafo)
```

Liberta a memória alocada para a lista de antenas e suas arestas.

Parameters

<i>lista</i>	Lista de antenas.
<i>grafo</i>	Grafo a ser limpo.

Returns

200 após a libertação completa.

Liberta a memória alocada para a lista de antenas e suas arestas.

Percorre a lista de antenas, libertando cada aresta e cada antena. Também limpa o grafo, definindo todos os ponteiros de antena como NULL.

Parameters

<i>lista</i>	Lista ligada de antenas
<i>grafo</i>	Grafo onde as antenas estão registradas

Returns

200 em caso de sucesso

Definition at line 450 of file [funcoes.c](#).

4.5.3.6 ImprimirMatriz()

```
int ImprimirMatriz (  
    Grafo * grafo,  
    int linhas,  
    int colunas)
```

Imprime a matriz com a posição das antenas.

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso.

Imprime a matriz com a posição das antenas.

A matriz representa:

- Cada antena com a sua frequência (ex: '0', 'A', etc.);
- Cada zona nefasta com o símbolo '#';
- Células vazias com o símbolo '.'.

Parameters

<i>grafo</i>	Ponteiro para o grafo que contém as antenas.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso.

Definition at line 32 of file [apresenta.c](#).

4.5.3.7 IniciarDFS()

```
int IniciarDFS (  
    Grafo * grafo,  
    int idOrigem)
```

Inicia a travessia DFS a partir de uma antena de origem.

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>idOrigem</i>	ID da antena de origem.

Returns

200 se bem-sucedido, 500 se o ID for inválido.

Inicia a travessia DFS a partir de uma antena de origem.

Verifica a validade da antena de origem e inicia a travessia.

Parameters

<i>grafo</i>	Grafo onde se realiza a travessia
<i>idOrigem</i>	ID da antena de origem

Returns

200 em caso de sucesso, 500 se o ID for inválido

Definition at line 277 of file [funcoes.c](#).

4.5.3.8 InserirAntena()

```
Ant * InserirAntena (  
    Ant * lista,  
    Grafo * grafo,  
    char freq,  
    int x,  
    int y,  
    int id)
```

Insere uma nova antena na lista e no grafo.

Parameters

<i>lista</i>	Lista atual de antenas.
<i>grafo</i>	Ponteiro para o grafo.
<i>freq</i>	Carácter da frequência da antena.
<i>x</i>	Coordenada X da antena.
<i>y</i>	Coordenada Y da antena.
<i>id</i>	Identificador único da antena.

Returns

Ponteiro para a nova cabeça da lista ligada.

Insere uma nova antena na lista e no grafo.

Parameters

<i>lista</i>	Lista ligada atual de antenas
<i>grafo</i>	Grafo onde a antena será registada pelo seu ID
<i>freq</i>	Carácter representativo da frequência da antena
<i>x</i>	Coordenada X (coluna) da antena
<i>y</i>	Coordenada Y (linha) da antena
<i>id</i>	Identificador único da antena

Returns

Ponteiro para a nova cabeça da lista ligada

Definition at line 104 of file [funcoes.c](#).

4.5.3.9 LerLista()

```
Ant * LerLista (
    const char * nomeFicheiro,
    const char * tipoFicheiro,
    int * linhas,
    int * colunas,
    Grafo * grafo)
```

Lê um ficheiro com a matriz de antenas e cria a lista ligada e o grafo correspondente.

Parameters

<i>nomeFicheiro</i>	Nome base do ficheiro (sem extensão).
<i>tipoFicheiro</i>	Extensão do ficheiro (".txt" ou ".bin").
<i>linhas</i>	Ponteiro onde será armazenado o número de linhas lidas.
<i>colunas</i>	Ponteiro onde será armazenado o número de colunas lidas.
<i>grafo</i>	Ponteiro para a estrutura do grafo a ser preenchida.

Returns

Ponteiro para o início da lista ligada de antenas.

Lê um ficheiro com a matriz de antenas e cria a lista ligada e o grafo correspondente.

Esta função percorre o ficheiro linha a linha e cria antenas para todas as células não vazias. Após a leitura, chama a função [CriarListaArestas\(\)](#) para estabelecer as ligações entre antenas.

Parameters

<i>nomeFicheiro</i>	Nome base do ficheiro (sem extensão)
<i>tipoFicheiro</i>	Extensão do ficheiro (".txt" ou ".bin")
<i>linhas</i>	Ponteiro para armazenar o número de linhas da matriz
<i>colunas</i>	Ponteiro para armazenar o número de colunas da matriz
<i>grafo</i>	Ponteiro para o grafo onde as antenas serão registadas

Returns

Ponteiro para o início da lista ligada de antenas

Definition at line 33 of file [funcoes.c](#).

4.5.3.10 ListarArestasENefastos()

```
int ListarArestasENefastos (  
    Grafo * grafo,  
    int linhas,  
    int colunas)
```

Lista todas as arestas existentes e suas zonas nefastas (se existirem).

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso.

Lista todas as arestas existentes e suas zonas nefastas (se existirem).

Para cada antena no grafo:

- Mostra a sua frequência e posição;
- Lista todas as arestas para antenas com mesma frequência;
- Indica a coordenada da zona nefasta associada, se válida;

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>linhas</i>	Número total de linhas da matriz (para validar nefastos).
<i>colunas</i>	Número total de colunas da matriz (para validar nefastos).

Returns

200 em caso de sucesso.

Definition at line 85 of file [apresenta.c](#).

4.5.3.11 TodosCaminhos()

```
int TodosCaminhos (
    Grafo * grafo,
    int idOrigem,
    int idDestino)
```

Inicia a busca por todos os caminhos possíveis entre duas antenas.

Parameters

<i>grafo</i>	Grafo com todas as antenas.
<i>idOrigem</i>	ID da antena de origem.
<i>idDestino</i>	ID da antena de destino.

Returns

200 em caso de sucesso.

Inicia a busca por todos os caminhos possíveis entre duas antenas.

Verifica se as antenas são válidas e têm a mesma frequência antes de iniciar a busca.

Parameters

<i>grafo</i>	Grafo com as antenas
<i>idOrigem</i>	ID da antena de origem
<i>idDestino</i>	ID da antena de destino

Returns

200 em caso de sucesso, ou se os IDs forem inválidos

Definition at line [403](#) of file [funcoes.c](#).

4.6 funcoes.h

[Go to the documentation of this file.](#)

```
00001
00013
00014 #pragma once
00015 #include <stdio.h>
00016 #include <stdlib.h>
00017 #include <string.h>
00018
00019 #define MAX_LINHAS 100
00020 #define MAX_COLUNAS 100
00021 #define MAX_VERTICES 200
00022
00023 #pragma region Estrutura_grafo
00024
00029 typedef struct Ant {
00030     char freqAntena;
00031     int id;
00032     int x;
00033     int y;
00034     struct Ant* proxAntena;
```

```

00035     struct Ars* listaAresta;
00036 } Ant;
00037
00042 typedef struct Ars {
00043     struct Ant* origemAntena;
00044     struct Ant* destinoAntena;
00045     struct Ars* proximaAresta;
00046     int xNef;
00047     int yNef;
00048 } Ars;
00049
00054 typedef struct Grafo {
00055     struct Ant* Antena[MAX_VERTICES];
00056 } Grafo;
00057
00058 #pragma endregion
00059
00060 #pragma region Leitura_Armazenamento
00061
00072 Ant* LerLista(const char* nomeFicheiro, const char* tipoFicheiro, int* linhas, int* colunas, Grafo*
    grafo);
00073
00085 Ant* InserirAntena(Ant* lista, Grafo* grafo, char freq, int x, int y, int id);
00086
00095 int CriarListaArestas(Ant* lista, int linhas, int colunas);
00096
00097 #pragma endregion
00098
00099 #pragma region DFS
00100
00109 int DFS(Ant* atual, int visitado[], int idOrigem);
00110
00118 int IniciarDFS(Grafo* grafo, int idOrigem);
00119
00120 #pragma endregion
00121
00122 #pragma region BFS
00123
00131 int BFS(Grafo* grafo, int idOrigem);
00132
00133 #pragma endregion
00134
00135 #pragma region Todos_Caminhos
00136
00147 int EncontrarCaminhos(Ant* atual, int idDestino, int visitado[], int caminho[], int posicao);
00148
00157 int TodosCaminhos(Grafo* grafo, int idOrigem, int idDestino);
00158
00159 #pragma endregion
00160
00161 #pragma region Impressao
00162
00171 int ImprimirMatriz(Grafo* grafo, int linhas, int colunas);
00172
00181 int ListarArestasENefastos(Grafo* grafo, int linhas, int colunas);
00182
00183 #pragma endregion
00184
00185 #pragma region Libertacao_Memoria
00186
00194 int FreeListaAntenas(Ant* lista, Grafo* grafo);
00195
00196 #pragma endregion

```

4.7 projetoEDA_F2_Exe/apresenta.c File Reference

Implementação de funções para apresentação de dados do grafo (matriz e lista de arestas).

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "funcoes.h"

```

Functions

- int [ImprimirMatriz](#) ([Grafo](#) *grafo, int linhas, int colunas)
Imprime a matriz de antenas e zonas nefastas.
- int [ListarArestasENefastos](#) ([Grafo](#) *grafo, int linhas, int colunas)
Lista todas as antenas e as respectivas arestas com as zonas nefastas associadas.

4.7.1 Detailed Description

Implementação de funções para apresentação de dados do grafo (matriz e lista de arestas).

Contém funções responsáveis por mostrar graficamente a matriz de antenas e os efeitos nefastos, bem como listar todas as antenas, suas ligações (arestas) e zonas nefastas associadas.

Author

Fábio Rafael Gomes Costa @contact a22997@alunos.ipca.pt @course Engenharia Sistemas Informáticos

Date

04/04/2025

Definition in file [apresenta.c](#).

4.7.2 Function Documentation

4.7.2.1 ImprimirMatriz()

```
int ImprimirMatriz (  
    Grafo * grafo,  
    int linhas,  
    int colunas)
```

Imprime a matriz de antenas e zonas nefastas.

Imprime a matriz com a posição das antenas.

A matriz representa:

- Cada antena com a sua frequência (ex: '0', 'A', etc.);
- Cada zona nefasta com o símbolo '#';
- Células vazias com o símbolo '.'.

Parameters

<i>grafo</i>	Ponteiro para o grafo que contém as antenas.
<i>linhas</i>	Número de linhas da matriz.
<i>colunas</i>	Número de colunas da matriz.

Returns

200 em caso de sucesso.

Definition at line [32](#) of file [apresenta.c](#).

4.7.2.2 ListarArestasENefastos()

```
int ListarArestasENefastos (
    Grafo * grafo,
    int linhas,
    int colunas)
```

Lista todas as antenas e as respectivas arestas com as zonas nefastas associadas.

Lista todas as arestas existentes e suas zonas nefastas (se existirem).

Para cada antena no grafo:

- Mostra a sua frequência e posição;
- Lista todas as arestas para antenas com mesma frequência;
- Indica a coordenada da zona nefasta associada, se válida;

Parameters

<i>grafo</i>	Ponteiro para o grafo.
<i>linhas</i>	Número total de linhas da matriz (para validar nefastos).
<i>colunas</i>	Número total de colunas da matriz (para validar nefastos).

Returns

200 em caso de sucesso.

Definition at line 85 of file [apresenta.c](#).

4.8 apresenta.c

[Go to the documentation of this file.](#)

```
00001
00013
00014 #include <stdio.h>
00015 #include <stdlib.h>
00016 #include <string.h>
00017 #include "funcoes.h"
00018
00032 int ImprimirMatriz(Grafo* grafo, int linhas, int colunas) {
00033     for (int y = 0; y < linhas; y++) {
00034         for (int x = 0; x < colunas; x++) {
00035             int marcado = 0;
00036
00037             // Verifica se existe uma antena nesta posição
00038             for (int i = 0; i < MAX_VERTICES && grafo->Antena[i] != NULL; i++) {
00039                 if (grafo->Antena[i]->x == x && grafo->Antena[i]->y == y) {
00040                     printf("%c ", grafo->Antena[i]->freqAntena);
00041                     marcado = 1;
00042                     break;
00043                 }
00044             }
00045
00046             if (marcado) continue; // Se já foi marcado, não precisa verificar os nefastos
00047
00048             // Verifica se existe um ponto nefasto nesta posição
00049             for (int i = 0; i < MAX_VERTICES && grafo->Antena[i] != NULL; i++) {
00050                 Ars* aresta = grafo->Antena[i]->listaAresta;
00051                 while (aresta != NULL) {
00052                     if (aresta->xNef == x && aresta->yNef == y) {
```



```

00053             printf("# ");
00054             marcado = 1;
00055             break;
00056         }
00057         aresta = aresta->proximaAresta;
00058     }
00059     if (marcado) break;
00060 }
00061
00062     if (!marcado) {
00063         printf(". ");
00064     }
00065 }
00066     printf("\n");
00067 }
00068     printf("\n\n");
00069     return 200;
00070 }
00071
00085 int ListarArestasENefastos(Grafo* grafo, int linhas, int colunas) {
00086     printf("\nLista de Antenas, Arestas e Zonas Nefastas:\n");
00087
00088     for (int i = 0; i < MAX_VERTICES; i++) {
00089         Ant* ant = grafo->Antena[i];
00090         if (ant == NULL)
00091             continue;
00092
00093         printf("\nAntena %d (freq. %c) em [%d, %d]:\n", ant->id, ant->freqAntena, ant->x, ant->y);
00094
00095         Ars* aresta = ant->listaAresta;
00096         int count = 0;
00097         if (aresta == NULL) {
00098             printf(" - Sem arestas.\n");
00099         }
00100
00101         while (aresta != NULL) {
00102             int xNef = aresta->xNef;
00103             int yNef = aresta->yNef;
00104             int nefastoValido = (xNef >= 0 && xNef < colunas && yNef >= 0 && yNef < linhas);
00105
00106             printf(" Aresta %d -> Antena %d (%c) em [%d, %d] | ",
00107                 count,
00108                 aresta->destinoAntena->id,
00109                 aresta->destinoAntena->freqAntena,
00110                 aresta->destinoAntena->x,
00111                 aresta->destinoAntena->y);
00112
00113             if (nefastoValido) {
00114                 printf("Zona nefasta: [%d, %d]\n", xNef, yNef);
00115             }
00116             else {
00117                 printf("Nao possui zona nefasta guardada, ou esta encontra-se fora dos limites da
matriz.\n");
00118             }
00119
00120             count++;
00121             aresta = aresta->proximaAresta;
00122         }
00123     }
00124     printf("\n\n");
00125     return 200;
00126 }

```

4.9 projetoEDA_F2_Exe/main.c File Reference

Ponto de entrada principal do programa que manipula grafos de antenas.

```

#include <stdio.h>
#include <stdlib.h>
#include "funcoes.h"

```

Functions

- int [main](#) ()

Função principal do programa.

4.9.1 Detailed Description

Ponto de entrada principal do programa que manipula grafos de antenas.

Este ficheiro contém o `main`, responsável por:

- Carregar as antenas a partir de um ficheiro;
- Imprimir a matriz de antenas e zonas nefastas;
- Listar todas as arestas e as zonas nefastas associadas;
- Realizar travessias DFS e BFS a partir de uma antena escolhida;
- Encontrar todos os caminhos entre duas antenas;
- Libertar a memória alocada dinamicamente no final.

Author

Fábio Rafael Gomes Costa

Date

18/05/2025

Definition in file [main.c](#).

4.9.2 Function Documentation

4.9.2.1 `main()`

```
int main ()
```

Função principal do programa.

Executa uma sequência de operações sobre um grafo de antenas, carregado a partir de um ficheiro. As operações incluem impressão da matriz, listagem de arestas, execução de DFS, BFS e descoberta de todos os caminhos entre dois nós. Ao final, toda a memória alocada é libertada.

Returns

int Retorna 0 se a execução for bem-sucedida, ou 1 em caso de erro.

Definition at line 30 of file [main.c](#).

4.10 main.c

[Go to the documentation of this file.](#)

```
00001
00016
00017 #include <stdio.h>
00018 #include <stdlib.h>
00019 #include "funcoes.h"
00020
00030 int main() {
00031     Grafo grafo;
00032     Ant* listaAntenas = NULL;
00033     int linhas = 0, colunas = 0;
00034     int validacaoResultado = 0;
00035
00036     // Leitura da matriz do ficheiro "antenas.txt"
00037     listaAntenas = LerLista("antenas", ".txt", &linhas, &colunas, &grafo);
00038     if (listaAntenas == NULL) {
00039         perror("Erro ao carregar a lista de antenas.\n");
00040         return 1;
00041     }
00042
00043     // Imprime a matriz com antenas e zonas nefastas
00044     validacaoResultado = ImprimirMatriz(&grafo, linhas, colunas);
00045     if (validacaoResultado != 200) {
00046         perror("Erro ao imprimir a matriz.\n");
00047     }
00048
00049     // Lista as arestas e zonas nefastas
00050     validacaoResultado = ListarArestasENefastos(&grafo, linhas, colunas);
00051     if (validacaoResultado != 200) {
00052         perror("Erro ao imprimir lista.\n");
00053     }
00054
00055     // Executa a procura em profundidade (DFS) a partir da antena 3
00056     validacaoResultado = IniciarDFS(&grafo, 3);
00057     if (validacaoResultado != 200) {
00058         perror("Erro na procura em profundidade.\n");
00059     }
00060
00061     // Executa a procura em largura (BFS) a partir da antena 3
00062     validacaoResultado = BFS(&grafo, 3);
00063     if (validacaoResultado != 200) {
00064         perror("Erro na procura em largura.\n");
00065     }
00066
00067     // Encontra todos os caminhos possíveis entre a antena 1 e 6
00068     validacaoResultado = TodosCaminhos(&grafo, 1, 6);
00069     if (validacaoResultado != 200) {
00070         perror("Erro na procura ao encontrar o caminho.\n");
00071     }
00072
00073     // Liberta toda a memória alocada
00074     validacaoResultado = FreeListaAntenas(listaAntenas, &grafo);
00075     if (validacaoResultado != 200) {
00076         perror("Erro ao libertar memória.\n");
00077     }
00078     else {
00079         printf("Memória libertada com sucesso.\n");
00080     }
00081
00082     return 0;
00083 }
```


Index

Ant, [5](#)
 freqAntena, [5](#)
 funcoes.h, [21](#), [32](#)
 id, [5](#)
 listaAresta, [6](#)
 proxAntena, [6](#)
 x, [6](#)
 y, [6](#)
Antena
 Grafo, [8](#)
apresenta.c
 ImprimirMatriz, [41](#)
 ListarArestasENefastos, [41](#)
Ars, [6](#)
 destinoAntena, [7](#)
 funcoes.h, [21](#), [32](#)
 origemAntena, [7](#)
 proximaAresta, [7](#)
 xNef, [7](#)
 yNef, [7](#)
BFS
 funcoes.c, [10](#)
 funcoes.h, [21](#), [32](#)
CriarListaArestas
 funcoes.c, [11](#)
 funcoes.h, [22](#), [32](#)
destinoAntena
 Ars, [7](#)
DFS
 funcoes.c, [11](#)
 funcoes.h, [22](#), [33](#)
EncontrarCaminhos
 funcoes.c, [12](#)
 funcoes.h, [23](#), [34](#)
FreeListaAntenas
 funcoes.c, [12](#)
 funcoes.h, [24](#), [34](#)
freqAntena
 Ant, [5](#)
funcoes.c
 BFS, [10](#)
 CriarListaArestas, [11](#)
 DFS, [11](#)
 EncontrarCaminhos, [12](#)
 FreeListaAntenas, [12](#)
 IniciarDFS, [13](#)
 InserirAntena, [13](#)
 LerLista, [14](#)
 TodosCaminhos, [14](#)
funcoes.h
 Ant, [21](#), [32](#)
 Ars, [21](#), [32](#)
 BFS, [21](#), [32](#)
 CriarListaArestas, [22](#), [32](#)
 DFS, [22](#), [33](#)
 EncontrarCaminhos, [23](#), [34](#)
 FreeListaAntenas, [24](#), [34](#)
 Grafo, [21](#), [32](#)
 ImprimirMatriz, [24](#), [35](#)
 IniciarDFS, [25](#), [36](#)
 InserirAntena, [26](#), [36](#)
 LerLista, [26](#), [37](#)
 ListarArestasENefastos, [27](#), [38](#)
 MAX_COLUNAS, [20](#), [31](#)
 MAX_LINHAS, [20](#), [31](#)
 MAX_VERTICES, [21](#), [31](#)
 TodosCaminhos, [28](#), [38](#)
Grafo, [8](#)
 Antena, [8](#)
 funcoes.h, [21](#), [32](#)
id
 Ant, [5](#)
ImprimirMatriz
 apresenta.c, [41](#)
 funcoes.h, [24](#), [35](#)
IniciarDFS
 funcoes.c, [13](#)
 funcoes.h, [25](#), [36](#)
InserirAntena
 funcoes.c, [13](#)
 funcoes.h, [26](#), [36](#)
LerLista
 funcoes.c, [14](#)
 funcoes.h, [26](#), [37](#)
listaAresta
 Ant, [6](#)
ListarArestasENefastos
 apresenta.c, [41](#)
 funcoes.h, [27](#), [38](#)
main
 main.c, [44](#)
main.c

- main, [44](#)
- MAX_COLUNAS
 - funcoes.h, [20](#), [31](#)
- MAX_LINHAS
 - funcoes.h, [20](#), [31](#)
- MAX_VERTICES
 - funcoes.h, [21](#), [31](#)
- origemAntena
 - Ars, [7](#)
- projetoEDA_F2/funcoes.c, [9](#), [15](#)
- projetoEDA_F2/funcoes.h, [19](#), [29](#)
- projetoEDA_F2_Exe/apresenta.c, [40](#), [42](#)
- projetoEDA_F2_Exe/funcoes.h, [30](#), [39](#)
- projetoEDA_F2_Exe/main.c, [43](#), [45](#)
- proxAntena
 - Ant, [6](#)
- proximaAresta
 - Ars, [7](#)
- TodosCaminhos
 - funcoes.c, [14](#)
 - funcoes.h, [28](#), [38](#)
- x
 - Ant, [6](#)
- xNef
 - Ars, [7](#)
- y
 - Ant, [6](#)
- yNef
 - Ars, [7](#)